

Explainability of the Quantum Layer in Physics-Informed Neural Networks: A Fourier-Bandwidth Analysis of Quantum-Assisted PINNs (QAPINN)

Team SpectralQ

WISER $\times$ BQP Global Quantum+AI 2026 Challenge

Shravan Kumar Sharma *University of Notre Dame, United States*

Mayank Sharma *Indian Institute of Technology Dharwad, India*

Satyabrat Sahu *Guru Gobind Singh Indraprastha University, Delhi, India*

7 August 2026

Abstract

Physics-Informed Neural Networks (PINNs) solve partial differential equations by penalizing the residual of the governing equation directly, instead of fitting labeled data. This report studies a structural variant, the Quantum-Assisted PINN (QAPINN), in which the first hidden layer of a classical PINN is replaced by a Variational Quantum Circuit (VQC). We ground the study in a specific, testable mechanism: a data-re-uploading VQC realizes a truncated multivariate Fourier series whose maximum harmonic order is fixed by the circuit’s structural parameters (qubit count, re-upload depth, and encoding structure) independently of training, while trainable encoding scales can change the physical locations of those harmonics while the Fourier coefficients are set by the trainable gates and the measurement operator. We use this fact to predict, rather than merely observe, when replacing a PINN’s first layer with a VQC helps or hurts, and we test that prediction on two canonical benchmarks: the one-dimensional heat equation (linear, closed-form solution, a controlled low-frequency target) and the one-dimensional viscous Burgers equation (nonlinear, shock-forming, a broadband high-frequency target). We report a complete, statistically validated comparison of a classical PINN, a classical periodic-activation control, and a QAPINN on both problems across three random seeds; a bandwidth sweep that locates the predicted accuracy elbow at the target spectrum’s highest wavenumber; and a spectral-complexity diagnostic testing whether the QAPINN’s smaller parameter count corresponds to genuinely lower model complexity. Along the way we describe, in detail, an apparent quantum advantage we initially measured on Burgers, traced it to an under-trained classical baseline, corrected it, and report the corrected result rather than the more favorable early one. The final result is a tie on the low-frequency problem, a clear classical win on the broadband one, and a concrete, falsifiable diagnostic for assessing, before full-scale training, whether a quantum layer’s accessible bandwidth is well matched to a given PDE.

Contents

1	Introduction	2
1.1	Related work	3
2	Theoretical Framework: A VQC as a Truncated Fourier Series	3
2.1	Data encoding and generator eigenvalues	3
2.2	Bandwidth from re-uploading	4
2.3	Practical consequence	5

3	Benchmark Problems	5
3.1	Heat equation	5
3.2	Burgers equation	5
3.3	Rationale for this pair	6
4	Model Architecture	6
4.1	Classical PINN	6
4.1.1	Sine-activation control	6
4.2	Quantum layer	6
4.3	QAPINN	7
5	Training Methodology	7
5.1	Loss function	7
5.2	Optimization schedule	7
5.3	Reproducibility	8
5.4	Compute environment	8
6	Experimental Design	9
6.1	Baseline comparison	9
6.2	Heat bandwidth sweep	9
6.3	Burgers bandwidth ladder	9
6.4	Spectral-complexity diagnostic	10
6.5	Training recipes	10
7	Results	10
7.1	The final scoreboard	10
7.2	Statistical significance	11
7.3	Heat bandwidth sweep confirms the theory	11
7.4	Burgers bandwidth ladder does not show the same pattern	12
7.5	Correcting an apparent quantum advantage	12
7.6	Spectral complexity and parameter efficiency	12
7.7	Timing	13
7.8	A practical recipe	13
8	Explainability Analysis	13
8.1	Method	14
8.2	Result: automated verification	14
8.3	Reference-solution validation	14
9	Limitations	14
10	Future Work	15
11	Conclusion	15
A	Configuration Reference	16

1 Introduction

A Physics-Informed Neural Network is trained not by fitting labeled examples but by minimizing the residual of a governing differential equation evaluated through the network via automatic differentiation,

together with a supervised term that enforces initial and boundary conditions. This gives a mesh-free surrogate for the PDE’s solution without requiring a numerical solver at inference time.

This report investigates a narrow, mechanistic question about PINN architecture: what changes, and why, when the network’s classical input projection is replaced by a small Variational Quantum Circuit (VQC)? We refer to the resulting model as a Quantum-Assisted PINN, or QAPINN. The resulting QAPINN uses a classical downstream network with the same nominal depth setting, but the quantum layer changes the dimensionality of the representation passed to that network. Consequently, the QAPINN has a different classical parameter count from the baseline, so the experiments test the combined effect of the quantum representation and the resulting architectural capacity difference rather than isolating the quantum layer alone.

We organize the investigation around three questions, matching the objective of the WISER $\times$ BQP Global Quantum+AI 2026 Challenge:

- (i) **Why** does replacing the first layer change training dynamics and final accuracy?
- (ii) **How** should the quantum circuit be designed, meaning qubit count, depth, encoding, entanglement, and measurement, for a given class of differential equations?
- (iii) **What** is gained and what is lost, concretely, by making this substitution?

1.1 Related work

Benchmarking a quantum-assisted PINN is not new on its own: Shah, Lineswala, and Chopra recently reported one for computational fluid dynamics [6], against a backdrop of an established variational quantum algorithm survey literature [8] and a sober assessment of what near-term, noisy hardware can and cannot yet do [10]. What distinguishes this report is the mechanism, not the benchmark: we derive a falsifiable prediction from the Fourier structure of a data-re-uploading circuit [2, 7] and use it to design every experiment in Sections 6 through 8.

Our approach is to answer these questions from a specific, testable mechanism rather than from architecture search: a VQC that encodes an input feature through Pauli-rotation gates realizes a *truncated Fourier series* in that feature, with an accessible bandwidth fixed by the circuit’s structure. Section 2 develops this mechanism precisely. Section 3 justifies the two PDE benchmarks used to test it. Sections 4 and 5 describe the models and training procedure. Section 6 describes the experimental design, and Section 7 reports the completed, statistically validated results, including the correction described in the abstract. Section 8 reports explainability diagnostics that validate the mechanism directly, independent of any accuracy number. Section 9 states limitations plainly, and Section 10 lists remaining work.

2 Theoretical Framework: A VQC as a Truncated Fourier Series

2.1 Data encoding and generator eigenvalues

Consider a single qubit and a data-encoding gate of the form $R_Y(sz) = \exp(-i \frac{sz}{2} Y)$, where z is a scalar input feature, s is a (possibly trainable) scaling factor, and Y is the Pauli- Y operator with eigenvalues $\{+1, -1\}$. Diagonalizing Y , the gate acts as a phase $\exp(\mp isz/2)$ on its eigenbasis. When this gate is embedded in a circuit together with trainable rotations and entangling gates, and the circuit output is read out as an expectation value of an observable, every occurrence of a data-encoding gate for feature z contributes an integer multiple of a base frequency to the Fourier expansion of the circuit’s output with respect to z [2]. Concretely, the map $z \mapsto f(z) = \langle \psi(z) | M | \psi(z) \rangle$ takes the form

$$f(z) = \sum_{n=-K}^K c_n e^{insz}, \quad (1)$$

where K denotes the maximum harmonic order relative to a common encoding scale s . When the encoding scales are fixed and shared, the reachable frequency set $\{-K, \dots, K\}$ is determined by the circuit structure, while the coefficients c_n are determined by the trainable rotations, entangling structure, and measurement operator. In configurations with trainable encoding scales, the structural harmonic ceiling remains fixed, but the physical locations of the corresponding frequencies can move as the scales are optimized.

2.2 Bandwidth from re-uploading

Encoding the feature once per qubit gives K equal to the number of qubits assigned to that feature. Data re-uploading, meaning re-encoding the same feature across successive circuit layers rather than only once at the start, increases the accessible harmonic order without adding qubits, and is itself a technique with its own dedicated literature on universal function approximation with a single re-uploaded qubit [7]. In this project’s implementation, the accessible bandwidth is most precisely defined per input feature. For feature z , let n_z denote the number of qubits assigned to that feature. Then

$$K_z = n_z \times n_{\text{uploads}}, \quad n_{\text{uploads}} = \begin{cases} n_{\text{layers}} & \text{encoding} = \text{reupload} \\ 1 & \text{encoding} = \text{angle}. \end{cases} \quad (2)$$

In Equation 2, K_z denotes the maximum harmonic index in the Fourier expansion with respect to feature z , not an absolute physical wavenumber. The corresponding frequencies are ns when a common encoding scale s is used. Thus, when the encoding scale is trainable, K_z describes the number of accessible harmonics while the scale determines their physical frequency spacing.

In the two-dimensional experiments where qubits are assigned evenly across the input features, this reduces to

$$K = \frac{n_{\text{qubits}}}{d_{\text{in}}} \times n_{\text{uploads}}. \quad (3)$$

Here $d_{\text{in}} = 2$ for the space-time coordinate (x, t) used throughout this work, and qubits are assigned to input features round robin. Equation 2 is the general per-feature bandwidth definition, while Equation 3 is the balanced form used for the four-qubit bandwidth-controlled Heat experiments in Section 6.2.

Proposition 1 (Bandwidth ceiling). *For a fixed circuit structure (qubit count, re-upload depth, entanglement pattern), no choice of trainable parameters can make the circuit’s output, as a function of an encoded feature z , contain Fourier content above harmonic K_z as given by Equation 2. Training can only redistribute weight among the coefficients $\{c_{-K_z}, \dots, c_{K_z}\}$, and cannot create new harmonic indices beyond that structural ceiling.*

Proposition 1 is the central, falsifiable claim this report tests. One boundary on its reach must be stated immediately, because it governs how the experimental results in Section 7.3 should be read: the proposition bounds the *quantum layer’s* output, not the QAPINN’s. The QAPINN feeds that layer into a tanh multilayer perceptron (Section 4), and a nonlinearity applied to a band-limited signal can synthesize harmonics above its input’s band. The full network is therefore not band-limited at K even though the circuit is, and a QAPINN whose circuit cannot reach a required mode is not thereby prevented from approximating it. What Proposition 1 predicts is that such a model must reconstruct the missing content indirectly, through the classical tail, and should pay for it in accuracy relative to a configuration whose circuit reaches the mode directly. Section 7.3 reports exactly that, and quantifies how much of the missing mode the tail recovers. If the target’s frequency content sits well inside K , replacing the classical first layer should cost little, and because the resulting representation is a fixed, structured basis rather than an arbitrary learned one, may impose a useful inductive bias on problems whose solution is genuinely band-limited or dominated by a small number of frequencies.

2.3 Practical consequence

Because Equation 2 lets us predict K before running any experiment, it converts the question of whether a quantum layer helps from a black-box accuracy comparison into a mechanistic, falsifiable prediction: accuracy should generally improve as the circuit bandwidth reaches the target’s dominant harmonic, while increasing K beyond the target’s relevant content does not guarantee further improvement. Section 6.2 operationalizes this as a controlled sweep, and Section 8 reports a direct, non-accuracy-based test of Equation 2 using empirical spectrum recovery.

3 Benchmark Problems

Two canonical one-dimensional, time-dependent PDEs are used, chosen to sit at opposite ends of the spectral-difficulty axis that Section 2’s theory makes predictions about.

3.1 Heat equation

$$u_t = \alpha u_{xx}, \quad x \in [0, 1], \quad t \in [0, T], \quad u(0, t) = u(1, t) = 0, \quad (4)$$

with initial condition given as an explicit finite sum of sine modes,

$$u(x, 0) = \sum_k b_k \sin(k\pi x), \quad (\text{default: } k \in \{1, 4\}, \quad b_1 = 1.0, \quad b_4 = 0.5). \quad (5)$$

Because sine modes are eigenfunctions of the Laplacian under these boundary conditions, the exact solution is available in closed form,

$$u(x, t) = \sum_k b_k e^{-\alpha(k\pi)^2 t} \sin(k\pi x), \quad (6)$$

For the bandwidth experiments, K is interpreted as a harmonic index in the normalized input coordinate and is compared with the target mode index under the encoding-scale convention used by the sweep. Thus, the statement that the $k = 4$ target mode is reached at $K = 4$ refers to the specific normalization and encoding convention of the implemented experiment, rather than asserting that K and the physical PDE wavenumber are universally identical quantities. giving a reference with no discretization error and, critically, a target whose exact frequency content is known in advance (only $k = 1$ and $k = 4$ are present). This makes the heat equation the controlled experiment for this study: any accuracy gap between a QAPINN and its classical counterpart can be checked directly against the circuit’s predicted bandwidth K (Equation 2), with no confound from a numerical reference solver.

3.2 Burgers equation

$$u_t + u u_x = \nu u_{xx}, \quad x \in [-1, 1], \quad t \in [0, 1], \quad u(-1, t) = u(1, t) = 0, \quad (7)$$

with initial condition $u(x, 0) = -\sin(\pi x)$ and viscosity $\nu = 0.01/\pi$ by default. The nonlinear advection term produces a sharp, near-discontinuous shock as t increases, a broadband, high-frequency feature and a substantially harder target than the heat equation’s smooth decay. No closed-form solution exists, so a high-accuracy numerical reference is constructed once via a Fourier pseudo-spectral method with integrating-factor RK4 time-stepping on a periodic grid, cached to disk after the first solve.

3.3 Rationale for this pair

The two problems isolate one variable, target spectral content, while holding the rest of the modelling setup fixed. The heat equation removes reference-solver noise entirely and gives an exactly known target spectrum. Burgers adds nonlinearity and a genuinely hard, broadband feature without changing anything else about the experimental pipeline. Both are standard PINN benchmarks since Raissi and colleagues [1], so baseline numbers are checkable against an existing literature. Harder or higher-dimensional PDEs (Navier-Stokes, the wave equation) were not pursued given the CPU-bound cost of quantum circuit simulation documented in Section 5.4: they would add solver complexity without adding new evidence about the mechanism under test.

4 Model Architecture

4.1 Classical PINN

The classical baseline is a fully connected network mapping normalized coordinates $(x, t) \in [-1, 1]^2$ to the scalar field u :

$$\text{MLP} : \quad \text{normalize} \rightarrow [\text{Linear}(2, H) \rightarrow \sigma] \rightarrow [\text{Linear}(H, H) \rightarrow \sigma]^{\times(D-1)} \rightarrow \text{Linear}(H, 1), \quad (8)$$

with $H = 20$ and $D = 4$ in every experiment reported here, and $\sigma \in \{\tanh, \sin, \text{ReLU}\}$. Weights are initialized with Xavier-normal initialization for tanh and ReLU. Inputs are affinely normalized to $[-1, 1]$ per dimension using the PDE’s domain bounds prior to the first layer, which we found necessary for stable training in preliminary runs.

4.1.1 Sine-activation control

When $\sigma = \sin$, the network uses the same Xavier-normal initialization as the tanh baseline. We did not implement the first-layer frequency multiplier $\omega_0 = 30$ or the layer-dependent uniform initialization that together define a SIREN proper [3]; the activation is $\sin(x)$ with no frequency scaling. We refer to this configuration as the “SIREN control” throughout, for continuity with the configuration filenames and the code, but it should be read as a plain sine-activation multilayer perceptron.

This does not weaken its role in the comparison. It remains a classical periodic-activation control trained under an identical budget, which is what Section 7 uses it for: to test whether any advantage of the quantum layer is really about the circuit or merely about having a periodic nonlinearity. It does mean the results here are not evidence about SIREN as published, and that a correctly initialized SIREN would likely be a *stronger* baseline than the one measured, which would strengthen rather than weaken this report’s conclusion.

4.2 Quantum layer

The quantum layer is implemented as a differentiable `torch.nn.Module` wrapping a PennyLane `QNode` on the `default.qubit` state-vector simulator, using the `backprop` differentiation method so that it supports the second-order input derivatives a PDE residual requires, for example u_{xx} . Table 1 lists every exposed design parameter. Per encoding layer $\ell = 1, \dots, n_{\text{layers}}$, the circuit applies a data-encoding rotation $R_Y(s_{\ell,q} z_q)$ on each qubit q assigned to input feature z_q , when $\ell = 1$ or when `encoding=reupload`; a trainable rotation $R_Z(\theta)R_Y(\theta)$ on every qubit; and an entangling layer of CNOTs following the chosen pattern. The circuit is measured either as per-qubit Pauli-Z expectation values (bounded, low-dimensional, computationally cheap) or as the full computational-basis probability distribution (exponentially larger in n_{qubits} , richer, and correspondingly costlier).

Table 1: Design parameters of the quantum layer.

Parameter	Role
n_{qubits}	register width; more qubits touching a feature raises its bandwidth K
n_{layers}	ansatz depth, number of data re-uploads
encoding	angle (upload once) or reupload (upload every layer)
entanglement	none , linear , ring , all_to_all (CNOT pattern)
measurement	expectation (n_{qubits} Pauli- Z values) or probs ($2^{n_{\text{qubits}}}$ basis probabilities)
trainable_scaling	whether the input-to-angle frequency multiplier s is learned

4.3 QAPINN

The QAPINN model replaces the classical input projection with the quantum layer of Section 4.2. With the depth setting used in this study, the resulting architecture is

$$\begin{aligned}
 \text{QAPINN : } & \text{normalize} \rightarrow \text{QuantumLayer}(2 \rightarrow Q) \\
 & \rightarrow [\text{Linear}(Q, H) \rightarrow \sigma]^{\times(D-2)} \rightarrow \text{Linear}(H, H) \rightarrow \sigma \\
 & \rightarrow \text{Linear}(H, 1).
 \end{aligned} \tag{9}$$

For the reported depth setting $D = 4$, the implementation contains three classical hidden blocks after the quantum layer: one $\text{Linear}(Q, H)$ block followed by two $\text{Linear}(H, H)$ blocks. The classical baseline instead contains the initial $\text{Linear}(2, H)$ projection followed by the full sequence of classical hidden blocks. Thus the QAPINN has fewer trainable parameters than the classical baseline in the reported configuration.

For expectation-value readout, $Q = n_{\text{qubits}}$; for probability readout, $Q = 2^{n_{\text{qubits}}}$. In the four-qubit expectation-value configuration, the resulting parameter counts are 985 for the QAPINN and 1,341 for the classical baseline. This difference is an architectural property of the implemented models and should not be interpreted as a parameter reduction caused by the quantum layer alone.

5 Training Methodology

5.1 Loss function

Training minimizes a weighted sum of the PDE-residual loss and a supervised initial and boundary condition loss,

$$\mathcal{L} = w_{\text{pde}} \cdot \frac{1}{N_c} \sum_{i=1}^{N_c} [\mathcal{R}(u_\theta; x_i, t_i)]^2 + w_{\text{data}} \cdot \frac{1}{N_s} \sum_{j=1}^{N_s} [u_\theta(x_j, t_j) - u_j^*]^2, \tag{10}$$

where $\mathcal{R}$ is the PDE residual evaluated through the network via automatic differentiation at N_c interior collocation points, and $\{(x_j, t_j, u_j^*)\}_{j=1}^{N_s}$ are Dirichlet initial and boundary samples. Both weights are set to 1.0 in every experiment reported here.

5.2 Optimization schedule

Training proceeds in two phases: an Adam warm-up for a fixed number of epochs, followed by an L-BFGS refinement phase using a strong-Wolfe line search. Algorithm 1 summarizes the procedure as implemented.

Algorithm 1 Training loop, Adam warm-up followed by L-BFGS refinement

```
1: Draw collocation set  $\{x_i, t_i\}_{i=1}^{N_c}$  and supervised set  $\{x_j, t_j, u_j^*\}_{j=1}^{N_s}$ 
2: for epoch = 1 to  $E_{\text{adam}}$  do
3:   if resample_every > 0 and due then
4:     Redraw the point sets
5:   end if
6:   Compute  $\mathcal{L}$ , backpropagate, take one Adam step
7: end for
8: if  $E_{\text{lbgfs}} > 0$  then
9:   Run one torch.optim.LBFGS closure with max_iter =  $E_{\text{lbgfs}}$ , history size 50, strong-Wolfe line
   search
10: end if
11: return trained model, training history, wall-clock and CPU time
```

5.3 Reproducibility

Every experiment is driven by a YAML configuration file specifying the PDE, model, and training hyperparameters, together with a single integer seed that seeds Python’s, NumPy’s, and PyTorch’s random number generators, with deterministic cuDNN algorithms requested. No hyperparameter is hard coded outside these configuration files. Re-running a configuration with the same seed on the same machine reproduces the reported numbers up to floating-point nondeterminism. The comparison in Section 7 uses three seeds per configuration (1234, 2025, 7) on one machine, so that differences between models can be checked against seed-to-seed variance rather than read off a single run.

5.4 Compute environment

Training was developed and primarily executed on CPU (PyTorch CPU execution, PennyLane `default.qubit` state-vector simulation), which is the reason qubit counts are kept modest: every trained model in this study uses a 2 to 5 qubit register, and the only 6-qubit circuit appearing anywhere is in the forward-only spectrum scan of Section 8, which does no training at all. Simulating an n -qubit statevector costs $O(2^n)$ in memory, and every PDE residual requires differentiating the circuit twice for second-order derivatives. Both wall-clock and CPU process time are recorded for every run, since wall-clock time can be corrupted by machine sleep or background load while process time is not; any wall-clock figure suspected of being an artifact is flagged explicitly rather than silently reported.

The training code also supports an explicit device setting (`auto`, `cpu`, or `cuda`), and PennyLane’s `default.qubit` simulator follows PyTorch tensors onto a CUDA device automatically under the `backprop` interface. We do not expect this to help the quantum layer itself at the qubit counts used here: a full statevector has at most $2^5 = 32$ amplitudes in any trained configuration, far too small for GPU parallelism to outperform CPU execution once host-device transfer and kernel-launch overhead are accounted for. For that reason the bandwidth sweep in Section 6.2 is deliberately pinned to CPU even when a GPU is present. A GPU is expected to help the classical tail and its autodiff on larger, non-quantum-layer-bound problems than the ones benchmarked here, but we have not yet run a direct, quantitative CPU-versus-GPU comparison; see Section 10.

For a heavier quantum simulation specifically, `pennylane-lightning[gpu]` is an option, but it only supports `diff_method="adjoint"`, which does not provide the second-order input derivatives a PDE residual needs. That makes it usable for forward-only analysis, for example large- K Fourier scans, but not for QAPINN training itself.

6 Experimental Design

6.1 Baseline comparison

For each PDE, a classical MLP, a sine-activation control (Section 4.1.1), and a 4-qubit QAPINN are trained under matched training recipes (identical collocation and supervised point counts, optimizer schedule, and loss weights). The QAPINN replaces the classical input projection with a quantum layer, but the resulting downstream representation and parameter count differ from the classical baseline. The comparison therefore evaluates the complete implemented architectures rather than an isolated first-layer substitution. Each configuration is trained at three random seeds, which is what makes the significance testing in Section 7 possible.

6.2 Heat bandwidth sweep

The heat equation’s target spectrum is exactly known (Section 3.1: wavenumbers $k \in \{1, 4\}$), which lets us design a direct test of Proposition 1. Using Equation 2, six $(n_{\text{qubits}}, \text{encoding}, n_{\text{layers}})$ combinations were chosen to realize exact target bandwidths $K \in \{1, 2, 3, 4, 5, 8\}$ (Table 2), each repeated over three random seeds, for 18 training runs in total. The prediction under Proposition 1: for $K < 4$, the circuit

Table 2: Bandwidth ladder for the heat K-sweep. Note that $K = 2$, $K = 4$, and $K = 8$ all use a four-qubit register; $K = 1$, $K = 3$, and $K = 5$ use a two-qubit register. Restricting to the four-qubit rows isolates K from register width, and is the slice reported as the headline result in Section 7.3.

K	n_{qubits}	encoding	n_{layers}	seeds
1	2	angle	1	3
2	4	angle	1	3
3	2	reupload	3	3
4	4	reupload	2	3
5	2	reupload	5	3
8	4	reupload	4	3

cannot reach the $k = 4$ mode directly, so error should be elevated relative to the $K = 4$ configuration; at $K = 4$, error should drop sharply once the target’s highest wavenumber becomes representable by the circuit itself; beyond $K = 4$, error may tick up slightly as the model’s added capacity permits mild overfitting.

It is worth being precise about what the sub-threshold error should *not* be expected to equal. Were the network band-limited at K as a whole, error below $K = 4$ would sit at the Parseval floor set by the energy in the unreachable mode, approximately 0.156 relative- L_2 for this initial condition. It does not, and it should not, for the reason given after Proposition 1: the classical tail is nonlinear and recovers much of the missing mode on its own. The Parseval value is therefore an upper bound on sub-threshold error, useful as a reference point, rather than a level the measurements should converge to. Section 7.3 reports where they actually land. A clean drop landing exactly at $K = 4$, rather than a gradual improvement with K , is the causal signature that bandwidth, not simply being quantum, explains the accuracy differences observed in the baseline comparison. Results are in Section 7.3.

6.3 Burgers bandwidth ladder

Burgers’ shock is a broadband feature rather than a small number of clean modes, so the relevant test is coarser: does the register width that minimizes error track any clean threshold, the way it does on Heat? QAPINN configurations at $n_{\text{qubits}} \in \{3, 4, 5\}$ (referred to as Q3, Q4, Q5) are trained under the identical Burgers training recipe (Table 3), at three seeds each. Results are in Section 7.4.

6.4 Spectral-complexity diagnostic

A smaller parameter count does not by itself imply a lower-complexity, less overfitting-prone model. To test this directly rather than assume it, we compute a spectral-complexity bound over each trained model’s classical **Linear** layers (the VQC contributes no weight matrix and is skipped automatically),

$$C = \left(\prod_{\ell} \|W^{(\ell)}\|_2 \right) \left(\sum_{\ell} \left(\frac{\|W^{(\ell)}\|_{2,1}}{\|W^{(\ell)}\|_2} \right)^{2/3} \right)^{3/2}, \quad (11)$$

where $\|W\|_2$ is the spectral (operator) norm and $\|W\|_{2,1} = \sum_j \|W_{:,j}\|_2$ is the sum of per-input-feature column ℓ_2 norms, following the group-norm construction used in spectrally-normalized generalization bounds [4], adapted to the physics-informed setting by Hu and colleagues’ study of when extended PINNs generalize [5]. If a QAPINN attains comparable or lower relative- L_2 error than its classical counterpart and a comparable or higher value of C , the claim that fewer parameters alone explain reduced overfitting is falsified for this problem.

6.5 Training recipes

Table 3 lists the recipe used for the final, statistically compared baseline runs reported in Section 7. The K-sweep (Section 6.2) uses a lighter recipe, 2,000 collocation points, 3,000 Adam epochs, no L-BFGS refinement, for tractability across 18 runs; this is a diagnostic sweep designed to locate the bandwidth elbow, not a head-to-head accuracy comparison, so it does not need the heavier budget used for the scoreboard. The L-BFGS budget of 5,000 steps is not incidental. Section 7.5 describes, in detail, why

Table 3: Final training recipe, used for every model in Section 7’s scoreboard.

	Heat recipe	Burgers recipe
Collocation points N_c	8,000	8,000
Supervised points N_s	400	400
Adam epochs	6,000	8,000
Adam learning rate	0.001	0.001
L-BFGS steps	5,000	5,000
Seeds	1234, 2025, 7	1234, 2025, 7

this specific number matters: an earlier version of this comparison used only 500 L-BFGS steps on the classical Burgers baseline, and correcting that single number reversed the project’s headline finding.

7 Results

7.1 The final scoreboard

Table 4 reports the complete, three-seed comparison between the classical PINN, the SIREN control, and the 4-qubit QAPINN on both PDEs, using the recipe in Table 3. Relative- L_2 error is computed against each PDE’s reference solution on a 256×100 evaluation grid. Seed-to-seed variance is reported as a coefficient of variation (standard deviation divided by mean) across the three seeds. Read plainly: on Heat, the quantum layer is close to, but slightly behind, both classical baselines. On Burgers, the classical baseline wins by a clear margin. In both cases the quantum layer trains far slower and lands with substantially more run-to-run variance. Section 7.2 quantifies whether the Heat gap is real.

Table 4: Final scoreboard. All rows use the recipe of Table 3; values are computed across seeds 1234, 2025, and 7.

Metric	Classical PINN	SIREN	QAPINN (4 qubits)
Heat, relative L_2 error	0.000243	0.000231	0.000487
Burgers, relative L_2 error	0.006268	0.014832	0.019406
Seed coefficient of variation	0.7% to 11%	21% to 36%	24% to 82%
Training time, Heat	2.7 min	2.4 min	60 min
Training time, Burgers	4.2 min	4.2 min	81 min
Trainable parameters	1,341	1,341	985

7.2 Statistical significance

We ran a two-sample t -test between the classical and QAPINN seed distributions on each PDE.

- **Heat:** $t = 1.05$. This does not clear significance. The gap between 0.000243 and 0.000487 in Table 4 is real in this sample, but with three seeds per arm and this much seed variance, we cannot conclude the quantum layer is actually worse here. We treat Heat as a statistical tie.
- **Burgers:** t between **4.8 and 4.82** across the two ways we computed it (Welch and pooled-variance forms). This clears significance. Classical wins on Burgers by roughly a factor of 3.1, and it is a repeatable result, not a lucky seed.

The one-sentence summary of the whole comparison: tied on the easy problem, beaten on the hard one, and roughly twenty times more expensive to train either way.

7.3 Heat bandwidth sweep confirms the theory

Restricting Table 2 to its three four-qubit rows ($K = 2, 4, 8$) isolates bandwidth from register width. Table 5 reports the result. The elbow lands exactly at $K = 4$, matching the heat solution’s real spectral

Table 5: Heat relative L_2 error versus bandwidth K , four-qubit register held fixed.

Bandwidth K	Relative L_2 error
$K = 2$	0.024
$K = 4$ (the target’s true content)	0.012
$K = 8$	0.016

content ($k = 1$ and $k = 4$), and error rises again once bandwidth exceeds what the target needs, consistent with the mild capacity cost predicted in Section 6.2. Section 8 confirms, with a direct spectral measurement rather than an accuracy proxy, that this ceiling is structural rather than a training artifact.

How much the classical tail recovers below threshold

Section 6.2 flagged that sub-threshold error should not be expected to reach the Parseval floor of approximately 0.156, because the network’s classical tail is nonlinear and can reconstruct content the circuit cannot reach. The full ladder quantifies how large that effect is:

	$K = 1$	$K = 2$	$K = 3$
Relative L_2 error	0.182	0.024	0.068
Fraction of Parseval floor	1.17	0.15	0.44

Only $K = 1$ sits at the floor. At $K = 2$ the network reaches 0.024, roughly one seventh of the floor, meaning the classical tail recovers the overwhelming majority of the $k = 4$ mode unaided. This is a genuine limit on how far Equation 2 can be pushed as a predictive tool: it correctly locates the elbow, and correctly orders $K = 2$ above $K = 4$, but it does not bound a QAPINN’s error from below, and any claim that a sub-threshold circuit is “provably” unable to fit the target would be wrong for this architecture. The circuit’s bandwidth sets where the model finds the target easy, not what it is capable of representing.

Register width, independent of bandwidth, still matters. In the full ladder of Table 2, $K = 5$ built from a two-qubit register scored roughly five times worse than $K = 4$ built from a four-qubit register. Equation 2 treats K as a single number, but bandwidth and register width are not interchangeable in practice, and a narrow register pushed to a high K through many re-uploads is not equivalent to a wider register reaching the same K with fewer re-uploads.

7.4 Burgers bandwidth ladder does not show the same pattern

The Q3, Q4, Q5 sweep described in Section 6 produced results that are statistically indistinguishable across seeds: no register width reliably beat any other. Unlike Heat’s clean two-mode target, a shock-forming, broadband solution is not well summarized by a single bandwidth number, which is itself an informative negative result. It tells us that Equation 2’s bandwidth ceiling is necessary but not sufficient for predicting quantum-layer behavior: it explains Heat well because Heat’s target genuinely lives at two clean frequencies, and it explains Burgers poorly because Burgers’ target does not.

7.5 Correcting an apparent quantum advantage

An earlier pass at the Burgers comparison, using the same architecture but only 500 L-BFGS refinement steps on the classical baseline, showed the QAPINN beating the classical PINN by roughly 7.8%. Before re-checking this result, we had already flagged an under-trained classical baseline as a named limitation in an internal draft of this report. Table 6 shows what happened once we tested for it directly. Properly

Table 6: Classical PINN, Burgers relative L_2 error, before and after correcting the L-BFGS budget.

Training configuration	Relative L_2 error
500 L-BFGS steps (original)	0.0756
5,000 L-BFGS steps (corrected)	0.0063

training the classical baseline improved it by a factor of 12, which is enough on its own to erase and reverse the apparent quantum advantage. The QAPINN’s Burgers error at the same, corrected recipe is 0.019406 (Table 4), more than three times worse than the properly trained classical baseline. The original 7.8% quantum win was a training-budget artifact on the classical side, not a real effect from the quantum layer. Every number reported in Sections 7 through 8 uses the corrected, 5,000-step baseline. We are including this correction in the body of the report, rather than only in a limitations footnote, because catching a comparison’s own training budget before reporting it is exactly the kind of check a benchmark like this needs, and we think the process is worth showing alongside the result.

7.6 Spectral complexity and parameter efficiency

Using Equation 11 on the trained checkpoints from Table 4, the QAPINN’s spectral-complexity bound comes out at roughly 3.2 times lower than the classical PINN’s, alongside its 27% smaller parameter count (985 versus 1,341). These measurements indicate that the complete implemented QAPINN architecture has a smaller measured spectral-complexity bound and fewer trainable parameters in this study. They

do not, however, establish that the quantum layer alone defines a more compact function class, because the classical downstream architecture is not parameter-matched to the baseline.

That lower complexity is not automatically favorable, and two observations about it sit alongside one another without either establishing the other. The QAPINN carries the lower complexity bound, and it also carries the higher seed-to-seed variance, running two to seven times the classical baseline’s coefficient of variation on both PDEs (Table 4). It is tempting to join these with a causal claim, on the reasoning that a smaller hypothesis class is a harder one to land consistently. We do not make that claim, because nothing in this study tests it. What we have is two measurements over eight model-PDE pairs that happen to point the same direction, with no intervention isolating model capacity from everything else that differs between a variational circuit and a **Linear** layer, and no evidence excluding the more mundane explanation that optimizing through a simulated circuit is simply a harder optimization problem. The pairing should be read as an observed correlation worth following up, not a demonstrated mechanism; Section 10 lists what would be needed to separate the two.

7.7 Timing

The quantum layer costs roughly twenty times the training time of the classical baseline on both PDEs (Table 4), entirely from re-simulating the circuit, including its second-order derivatives, at every optimizer step. This is the dominant practical cost of the whole study and the reason qubit counts and epoch budgets are kept modest throughout. As discussed in Section 5.4, we do not expect a GPU to close this gap at the register widths used here; the bottleneck is per-gate execution overhead in the simulator, not the kind of dense linear algebra a GPU accelerates.

7.8 A practical recipe

Taken together, Sections 7.3 through 7.5 support a concrete checklist for deciding whether to add a quantum layer to a PINN for a new PDE, rather than a blanket yes or no answer.

1. Ask first whether a quantum layer is needed at all. A classical periodic-activation network may achieve the same result at a fraction of the training cost. Ours did on both PDEs, and it was only a sine-activation MLP (Section 4.1.1); a correctly initialized SIREN would be a stronger baseline still, not a weaker one.
2. Fourier-analyze the target before choosing hardware. Know its true wavenumbers, the way Heat’s are known to be exactly $k = 1$ and $k = 4$.
3. Check that the target’s modes fit inside the corresponding per-feature bandwidths K_z with margin, not exactly at the boundary. $K = 4$ matching the target’s highest mode in the controlled Heat experiment, as in Section 7.3, is the least forgiving place to sit if variance matters, since none of the relevant harmonic capacity is held in reserve.
4. Bandwidth can be bought cheaply with data re-uploads, but register width still governs capacity independently, as shown by the $K = 5$ versus $K = 4$ comparison in Section 7.3.

Our own conclusion, stated plainly: a quantum layer is a constraint that has to be earned. On these two problems, at this scale, it was not.

8 Explainability Analysis

Accuracy comparisons alone cannot distinguish a change caused by the quantum layer’s Fourier bandwidth from any number of alternative explanations, such as optimization dynamics, initialization scale, or parameter count. We therefore verify Equation 2 directly, independent of any PDE or training run, by recovering a quantum layer’s empirical Fourier spectrum and comparing it against the bandwidth the equation predicts.

8.1 Method

For a fixed quantum layer configuration, one input feature is swept over exactly one period of its encoding while the other is held fixed, the layer’s output is recorded at each swept point, and a discrete Fourier transform is applied to the resulting sequence. This recovers the magnitude $|c_n|$ of every harmonic n the layer actually exhibits, which can be compared directly against the theoretical per-feature bandwidth K_z of Equation 2 for that configuration, with no training involved.

8.2 Result: automated verification

This check is implemented as a parametrized automated test (`tests/test_fourier.py`) rather than a one-off figure, so the bandwidth relationship is verified continuously rather than asserted once. Table 7 reports the four circuit configurations tested and the outcome. We also swept the untrained circuit’s

Table 7: Empirical versus theoretical bandwidth verification. In every case, the highest harmonic carrying non-negligible spectral weight (threshold: 1% of peak magnitude) exactly equals the predicted bandwidth for the encoded feature, with no significant weight observed above it.

n_{qubits}	n_{layers}	d_{in}	encoding	predicted = observed K
1	1	1	angle	1
1	3	1	reupload	3
2	1	1	angle	2
4	2	2	reupload	4

output through this same DFT procedure and confirmed its energy is exactly zero past K , for every configuration tested. This is the direct evidence behind Section 7.3’s claim that the bandwidth ceiling is a structural property of the circuit rather than something that only appears after training. Because K is independently verified to be exactly what the encoding predicts, the accuracy elbow at $K = 4$ in Section 7.3 cannot be explained away as coincidence.

8.3 Reference-solution validation

As a methodological control, the reference solutions used to compute every error metric in this report were themselves checked against their governing equations and boundary data. The closed-form heat solution was verified to satisfy the heat equation to a residual below 10^{-4} and to reproduce the specified initial condition to within 10^{-6} . The numerically constructed Burgers reference was verified to reproduce its initial condition to relative- L_2 error below 5×10^{-3} and its Dirichlet boundary values to within 10^{-3} (`tests/test_pdes.py`). All eight automated tests in the project’s test suite pass at the time of writing.

9 Limitations

We state the following limitations plainly rather than omit them, in the interest of an accurate record.

- (L1) **Simulator only.** Every circuit in this study ran on PennyLane’s state-vector simulator. We have no results yet from physical, near-term noisy quantum hardware [10], and none of the noise that comes with it.
- (L2) **The original eighteen-task ladder confounds qubit count with bandwidth.** Only the four-qubit slice used in Section 7.3 isolates K cleanly; the remaining rows of Table 2 mix register width with bandwidth and should be read as suggestive rather than conclusive on their own.
- (L3) **Readout scheme was mostly held fixed.** We compared expectation-value and probability readout only at a single configuration early in the project, where probability readout roughly

halved Heat’s error at a smaller training budget. A full sweep of this dial, at the final training recipe, has not been run; see Section 10.

- (L4) **Narrow scope.** All experiments use two one-dimensional PDEs and trained registers of only 2 to 5 qubits, at three seeds. This is not evidence about higher-dimensional PDEs, larger quantum registers, or physical hardware noise.
- (L5) **No barren-plateau measurement.** We did not measure how gradient variance scales with qubit count, a known risk once VQCs grow past the modest sizes used here, and one that has been shown to scale unfavorably with system size for generic circuit ansatze [9].
- (L6) **The classical periodic control is not a faithful SIREN.** As stated in Section 4.1.1, the sin configuration uses $\sin(x)$ with Xavier-normal initialization, not the ω_0 scaling and layer-dependent uniform initialization of [3]. It is a valid periodic-activation control at a matched budget, but results here are not evidence about SIREN as published.
- (L7) **Capacity and variance are correlated, not causally separated.** The spectral-complexity bound and the seed-to-seed variance point the same way across our eight model-PDE pairs, but no experiment here isolates capacity from the other differences between a VQC and a **Linear** layer.

10 Future Work

- A full sweep of readout scheme (expectation value versus probability) at the final training recipe, following up on the single promising result noted in limitation (L3).
- A K -margin sweep on Heat: does sitting comfortably inside K , for example $K = 6$ for a target that needs $K = 4$, reduce the seed-to-seed variance observed at the exact boundary in Section 7.3?
- A direct, quantitative CPU-versus-GPU timing comparison for a classical-only configuration, to quantify the benefit described qualitatively in Section 5.4.
- The nine-cell viscosity by qubit-count grid (already prepared as `burgers_nu` configs), to test whether the register width a shock needs tracks its width as viscosity changes.
- At least one configuration run on physical hardware or a noisy simulator, to see how much of the training-time and variance gap in Section 7 comes from ideal, noiseless simulation specifically.
- Barren-plateau variance measurement, loss-landscape visualization, and gradient-based input attribution, extending the explainability toolkit of Section 8.
- An experiment separating model capacity from optimization difficulty as explanations for the QAPINN’s seed variance: hold the circuit fixed and vary only the classical tail’s width to move the complexity bound without touching the quantum layer, then check whether variance tracks capacity or stays pinned to the presence of the circuit.
- A faithful SIREN baseline, with ω_0 scaling and the initialization of [3], to replace the sine-activation control of Section 4.1.1 and tighten the comparison in Section 7.8.

11 Conclusion

This report developed and tested a specific, falsifiable mechanism for why replacing a PINN’s first layer with a variational quantum circuit changes its accuracy: the resulting model computes a truncated Fourier series whose bandwidth is fixed by circuit structure and independent of training. That mechanism held up under direct, non-accuracy-based verification (Section 8), and it correctly predicted a sharp accuracy elbow at $K = 4$ on the heat equation, whose target spectrum is exactly known (Section 7.3).

It did not, on its own, explain Burgers. The broadband, shock-dominated target is not well summarized by a single bandwidth number, and the Q3 through Q5 sweep showed no consistent winner (Section 7.4). More importantly, our first measurement on Burgers pointed the wrong way entirely: an

apparent quantum advantage that, once we corrected an under-trained classical baseline, reversed into a clear classical win (Section 7.5).

The final, statistically validated comparison across three seeds is a tie on Heat and a loss on Burgers, at roughly twenty times the training cost and substantially higher run-to-run variance in both cases (Section 7). A quantum layer is a constraint that has to be earned. On these two problems, at this scale, it was not. The recipe an engineer can use on the next problem is the one in Section 7.8: compute the per-feature bandwidth $K_z = n_z \times n_{\text{uploads}}$, Fourier-analyze the target, check that the relevant modes fit inside the corresponding K_z values with margin, and then, going by the evidence in this report, use the classical network anyway.

References

- [1] M. Raissi, P. Perdikaris, and G. E. Karniadakis, “Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations,” *Journal of Computational Physics*, vol. 378, pp. 686–707, 2019.
- [2] M. Schuld, R. Sweke, and J. J. Meyer, “Effect of data encoding on the expressive power of variational quantum-machine-learning models,” *Physical Review A*, vol. 103, no. 3, p. 032430, 2021.
- [3] V. Sitzmann, J. N. P. Martel, A. W. Bergman, D. B. Lindell, and G. Wetzstein, “Implicit neural representations with periodic activation functions,” *Advances in Neural Information Processing Systems*, vol. 33, pp. 7462–7473, 2020.
- [4] P. L. Bartlett, D. J. Foster, and M. J. Telgarsky, “Spectrally-normalized margin bounds for neural networks,” *Advances in Neural Information Processing Systems*, vol. 30, 2017.
- [5] Z. Hu, A. D. Jagtap, G. E. Karniadakis, and K. Kawaguchi, “When do extended physics-informed neural networks (XPINNs) improve generalization?” *SIAM Journal on Scientific Computing*, vol. 44, no. 5, pp. A3158–A3182, 2022.
- [6] K. Shah, P. Lineswala, and H. Chopra, “Benchmarking quantum-assisted PINN for computational fluid dynamics,” in *Proceedings of the IEEE International Conference on Quantum Computing and Engineering (QCE)*, pp. 1707–1712, 2024.
- [7] A. Pérez-Salinas, A. Cervera-Lierta, E. Gil-Fuster, and J. I. Latorre, “Data re-uploading for a universal quantum classifier,” *Quantum*, vol. 4, p. 226, 2020.
- [8] M. Cerezo, A. Arrasmith, R. Babbush, S. C. Benjamin, S. Endo, K. Fujii, J. R. McClean, K. Mitarai, X. Yuan, L. Cincio, and P. J. Coles, “Variational quantum algorithms,” *Nature Reviews Physics*, vol. 3, no. 9, pp. 625–644, 2021.
- [9] J. R. McClean, S. Boixo, V. N. Smelyanskiy, R. Babbush, and H. Neven, “Barren plateaus in quantum neural network training landscapes,” *Nature Communications*, vol. 9, p. 4812, 2018.
- [10] J. Preskill, “Quantum computing in the NISQ era and beyond,” *Quantum*, vol. 2, p. 79, 2018.

A Configuration Reference

Every experiment reported here is fully specified by a YAML configuration file; none of the results in this report depend on any parameter not visible in its corresponding file. The configurations behind Table 4 live in `configs/rerun/`, one file per (model, seed) pair, and are distinct from the lighter exploratory configurations at the top level of `configs/`. The four-qubit heat QAPINN configuration used in Table 4 is reproduced below; its classical counterpart, `T1_heat_pinn_s1234.yaml`, is byte-identical apart from the `model:` block, which is what makes the comparison budget-matched.

Listing 1: `configs/rerun/T1_heat_q4_s1234.yaml`

```

1 run_name: rerun/heat_q4_s1234
2 seed: 1234
3 pde:
4   name: heat

```

```

5   alpha: 0.05
6   modes: [[1, 1.0], [4, 0.5]]
7   t_max: 1.0
8   model:
9     type: qapinn
10    hidden: 20
11    depth: 4
12    activation: tanh
13    quantum:
14      n_qubits: 4
15      n_layers: 2
16      encoding: reupload
17      entanglement: ring
18      measurement: expectation
19      trainable_scaling: true
20  training:
21    n_collocation: 8000
22    n_supervised: 400
23    adam_epochs: 6000
24    adam_lr: 0.001
25    lbfgs_epochs: 5000
26    resample_every: 0
27    w_pde: 1.0
28    w_data: 1.0
29    log_every: 500
30    eval_every: 1000
31    device: auto

```

AI-Tool Disclosure. AI coding assistants were used during this project as scaffolding and drafting aids for source code, configuration files, documentation, and report editing. The team independently developed and verified the mathematical derivations, experimental design, training methodology, and interpretation of the results, including the correction described in Section 7.5. AI tools were not used to fabricate, alter, or selectively report experimental results. All numerical results reported in this work were generated from the project code and configuration files and were independently checked by the team before submission. The team takes responsibility for the accuracy and integrity of the submitted methods, results, and conclusions.